Ejercicios: TLS y DTLS

- Requiere `OpenSSL`, certificados y claves privadas para el servidor y el cliente.
- `OpenSSL` proporciona una herramienta para ejecutar servidores seguros de protocolos de aplicación sencillos: `s_server`[1]. Por ejemplo: para lanzar un servidor `echo` seguro:

  `$ openssl s_server -accept` *`numport`* `-cert` *`certserver.pem`*
  `-key` *`keyserver.pem`* `[-cert_chain` *`chain-server.pem]`* *`-protocolo`*
  `protocolo` `= {tls1, tls1_1, tls1_2, tls1_3, dtls1, dtls1_2}`[2]
- También proporciona una herramienta para ejecutar clientes seguros: `s_client`[3]. Por ejemplo: para ejecutar un cliente `echo` seguro:

  `$ openssl s_client -connect` *`host:numport`* *`-protocolo`* `[-cipher RSA]`

  Para finalizar la conexión: `Q + enter`

Ejercicios

1. Realizar una captura de una sesión TLSv1.2 con Wireshark e identificar los mensajes TLSv1.2 transferidos.
2. ¿Qué diferencias puedes encontrar entre la captura de una sesión TLSv1.2 y una sesión TLSv1.3?
3. ¿Pueden interactuar entre sí ambos protocolos, TLSv1.2 y TLSv1.3?
4. Demostrar la carencia de la propiedad *Forward Secrecy* si se utiliza el algoritmo RSA.
5. Establece una sesión TLSv1.2 entre cliente y servidor que requiera la autenticación del cliente (mediante certificado). Identificar los nuevos mensajes TLS. ¿Podrían observarse estos mensajes en una sesión TLSv1.3?
6. Reutilizar una sesión TLSv1.2 (*session resumption*) entre cliente y servidor. Analizar las diferencias entre la sesión inicial y la reutilizada.
7. Establecer una sesión con *0-RTT Handshake*.
8. Realizar una captura de una sesión DTLSv1.2 con Wireshark. ¿Qué diferencias puedes encontrar entre la captura de una sesión con TLSv1.2 y otra con DTLSv1.2?
9. Analizar el uso de algoritmos PQC para el establecimiento de claves de sesión y autenticación del servidor en una sesión TLSv1.3

[1] Más información: `man s_server`. También en: `openssl s_server -help`
[2] `-ssl3` no disponible a partir de la versión 1.0.1;
`-tls1_3` disponible desde la versión 1.1.1(`$openssl version -a`)
[3] Más información: `man s_client`. También en: `openssl s_client -help`

ANEXO

- https://www.openssl.org/docs/manmaster/man1/openssl-s_client.html
- https://www.openssl.org/docs/manmaster/man1/openssl-s_server.html
- OpenSSL y Wireshark:
  - Capturar en la interfaz `loopback` (o seleccionar `any`)
  - Si el protocolo se denomina `PUSH`, descodificar las tramas, en el menú contextual: `Decoded As` –> `Transport` –> `TLS`
  - Para que `openssl` utilice RSA, añadir la opción `-cipher RSA` al lanzar el servidor/cliente. Para descifrar tramas capturadas necesita tener acceso a la clave privada del servidor: `Edit -> Preferences -> Protocols -> TLS`. Editar la lista de claves RSA, y añadir los datos sobre el servidor (*127.0.1.1* - *numport* - `data` - *serverkey.file* - [*password*]).
  - `Openssl`, por defecto utiliza *DH, para descifrar las tramas TLS capturadas necesitamos proporcionar a Wireshark todo el material criptográfico utilizado. El cliente de `openssl` exporta todo el material a un fichero con la opción `-keylogfile <`*filename*`>` al que puede acceder Wireshark desde `Edit -> Preferences -> Protocols -> TLS`, indicar el fichero en `(Pre)-Master-Secret log filename`.
  - Para realizar *session resumption* openssl facilita las opciones `-sess_out <`*filename*`>` y `-sess_in <`*filename*`>`
  - Para permitir `0-RTT Handshake` se utiliza la opción `-early_data`
  - Para ver el tráfico capturado, en la ventana de bytes o desde el menú `Analyze -> Follow -> TLS Stream`